TP4 : neural network - Part 1 -

We recall that a neural network with three layers can be modeled as follows:

- $a^{(1)} = X$
- $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$
- $a^{(2)} = \sigma(z^{(2)})$
- $z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}$
- $a^{(3)} = \sigma(z^{(3)})$

The parameters of the model are in the matrices and vectors $W^{(2)}$, $W^{(3)}$, for the weights and $b^{(2)}$, $b^{(3)}$ for the bias. The sigmoid function σ is defined by:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In the Stanford machine learning course, the weights and the bias of each layer l is contained in a single matrix denoted by $\Theta^{(l)}$. For each layer l, this matrix satisfies the equation

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} = \Theta^{(l)}\widetilde{a}^{(l-1)}$$

Question 1:

Specify the matrix $\Theta^{(l)}$ as a function of $W^{(l)}$ and $b^{(l)}$, and $\widetilde{a}^{(l-1)}$ as a function of $a^{(l)}$ such that the above relation be satisfied.
Helper: you can think of $\Theta^{(l)}$ having the following shape:

$$\Theta^{(l)} = \begin{pmatrix} \begin{pmatrix} & \\ & \end{pmatrix} & \begin{pmatrix} & \\ & \end{pmatrix} \end{pmatrix},$$

and you can look for $\widetilde{a}^{(l-1)}$ with the following form :

$$\widetilde{a}^{(l-1)} = \begin{pmatrix} 1 \\ \\ \end{pmatrix}$$

In the remaining of the tutorial, we will store the parameters into the matrices/vectors $(W^{(l)}, b^{(l)})$ instead of $\Theta^{(l)}$.

Question 2:

Justify why we should use a sigmoid function as activation function on the last layer when doing a binary classification problem.

At first, we will use the binary cross entropy as a loss (or cost) function for a binary classification problem. For a given input data x, this cost function is defined as

$$C = \frac{-1}{m} \sum_x y(x) \ln(a_x^{(3)}) + (1 - y(x)) \ln(1 - a_x^{(3)}),$$

where m is the size of the dataset, $a_x^{(3)}$ is the output of the neural network obtained for an input x and $y(x)$ is the expected true value for an input x. Note that for a binary classification problem, the output $y(x) \in \{0, 1\}$. We recall that the parameters of the model are the ones that minimize the cost function. Finding the minimum of the cost function can be performed through a gradient descent algorithm that require the computation of the derivatives of C with respect to the parameters of the model. The backpropagation algorithm is an efficient algorithm that computes those derivatives. We recall the vectorized version of the backpropagation algorithm, i.e. when the entire input dataset is stored into a matrix X whose columns are filled with the m observations x_i with $i \in [\![1, m]\!]$ and similarly with $y_i = y(x_i)$:

- $a^{(1)} = X = \begin{pmatrix} x_1 & \cdots & x_m \end{pmatrix}$ and $Y = \begin{pmatrix} y_1 & \cdots & y_m \end{pmatrix}$
- For $l = 2, 3$ compute
$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \text{ and } a^{(l)} = \sigma(z^{(l)})$$
- Compute
$$\Delta^{(3)} = \frac{\partial C}{\partial a^{(3)}} \odot \sigma'(z^{(3)})$$
- Compute
$$\Delta^{(2)} =^t W^{(3)} \Delta^{(3)} \odot \sigma'(z^{(2)})$$
- For $l = 2, 3$ compute
$$\frac{\partial C}{\partial W^{(l)}} = \frac{1}{m} \Delta^{(l)t} a^{(l-1)} \text{ and } \frac{\partial C}{\partial b^{(l)}} = \frac{1}{m} \sum_{i=1}^{m} \Delta_i^{(l)},$$

where $\Delta_i^{(l)}$ denote the column i of the matrix $\Delta^{(l)}$. Here we have assumed that all the parameters of the model are stored into $W^{(l)}$ and $b^{(l)}$. We recall that the backpropagation requires the computation of $\frac{\partial C}{\partial a^{(3)}}$, which is given by

$$\frac{\partial C}{\partial a^{(3)}} \underset{def}{=} \begin{pmatrix} \frac{\partial C_{x_1}}{\partial a^{(3)}} \\ \vdots \\ \frac{\partial C_{x_m}}{\partial a^{(3)}} \end{pmatrix} = \frac{(a^{(3)} - y)}{a^{(3)}(1 - a^{(3)})},$$

for the binary cross entropy cost function.

In this tutorial, the dataset is generated with sinus or exponential functions, i.e we start by drawing $u_i \sim U([0,1])$ and $x_i = \sin(2\pi u_i)$ for 50% of the data and $x_i = e^{u_i} - 1$ for the remaining data. The task of the neural network is to tell if the input data x_i comes from a sinus function or an exponential function.

Question 3:

We assume that the second layer contains n_2 neurons and the size of the learning dataset is m. Specify all the dimensions of the matrices/vectors appearing in the above backpropagation algorithm.

Question 4:

You are given a python code that implement the sinus/exponentiel classification. The parameters are set as follows:

$n_2 = 15$ (15 neurons in the second layer)
$m = 120$ (60 data coming from a sinus function + 60 coming from an exponential function)
$\alpha = 0.3$ (learning rate)
$Niter = 8000$ (number of steps in the gradient descent algorithm)

This code uses the mean square error as cost function, i.e.

$$C = \frac{1}{2m} \sum_x (a_x^{(3)} - y(x))^2$$

You must modify this code so that it uses the binary cross entropy instead.

Which part of the backpropagation algorithm must be modified? For the binary cross entropy, show that

$$\Delta^{(3)} = (a^{(3)} - y),$$

and make the necessary modifications to the given code.

So far, the output $a_x^{(3)} = \sigma(z_x^{(3)}) \in [0,1]$ and we have assumed that

$$\begin{cases} \text{if } a_x^{(3)} > 0.5, \text{ the input } x \text{ is generated with a sinus function} \\ \text{if } a_x^{(3)} \leq 0.5, \text{ the input } x \text{ is generated with an exponential function} \end{cases}$$

We now decide to code the output of the neural network in a different manner: instead of being a scalar, the output is a vector of $\mathbb{R}^2$. More precisely, $a_x^{(3)} = \sigma(z_x^{(3)}) \in [0,1] \times [0,1]$ and the classification decision is made as follows: assume that $a_x^{(3)} =^t \left((a_x^{(3)})_1, (a_x^{(3)})_2\right)$, then

$$\begin{cases} \text{if } (a_x^{(3)})_1 > (a_x^{(3)})_2, \text{ the input } x \text{ is generated with a sinus function} \\ \text{otherwise, the input } x \text{ is generated with an exponential function} \end{cases}$$

As a consequence, the dataset needs to be modified since we now have $y(x) =^t (1,0)$ if the input x is generated with a sinus function and $y(x) =^t (0,1)$ if the input x is generated with an exponential function.

Question 5:

What is the new size of the vector Y ? Modify the dataset accordingly.

The cost function also needs to be modified in order to take into account the two components of the output of the neural network. Denoting $y(x) =^t (y_1(x), y_2(x))$ The new cost function will be

$$\begin{aligned} C &= \frac{-1}{m}\sum_x y_1(x)\ln((a_x^{(3)})_1) + (1-y_1(x))\ln(1-(a_x^{(3)})_1) - \\ &\quad \frac{1}{m}\sum_x y_2(x)\ln((a_x^{(3)})_2) + (1-y_2(x))\ln(1-(a_x^{(3)})_2) \\ C &= \frac{-1}{m}\sum_{k=1}^{2}\sum_x y_k(x)\ln((a_x^{(3)})_k) + (1-y_k(x))\ln(1-(a_x^{(3)})_k) \end{aligned}$$

In other words, we should apply the binary cross entropy to the two outputs components of the neural network and sum them up. The part of the backpropagation algorithm that need to be modified is

$$\Delta^{(3)} = \frac{\partial C}{\partial a^{(3)}} \odot \sigma'(z^{(3)})$$

The new cost function can be written $C = C_1 + C_2$ with

$$C_1 = \frac{-1}{m}\sum_x y_1(x)\ln((a_x^{(3)})_1) + (1-y_1(x))\ln(1-(a_x^{(3)})_1)$$

and

$$C_2 = -\frac{1}{m}\sum_x y_2(x)\ln((a_x^{(3)})_2) + (1-y_2(x))\ln(1-(a_x^{(3)})_2)$$

Accordingly, the computation of $\Delta^{(3)}$ should be modified as

$$\Delta^{(3)} \underset{def}{=} \frac{\partial C}{\partial z^{(3)}} = \begin{pmatrix} \frac{\partial C}{\partial (a^{(3)})_1} \\ \frac{\partial C}{\partial (a^{(3)})_2} \end{pmatrix} \odot \begin{pmatrix} \sigma'((z^{(3)})_1) \\ \sigma'((z^{(3)})_2) \end{pmatrix},$$

where

$$z^{(3)} = \begin{pmatrix} (z^{(3)})_1 \\ (z^{(3)})_2 \end{pmatrix}.$$

Question 6:

Justify why you can write

$$\Delta^{(3)} = \begin{pmatrix} \frac{\partial C_1}{\partial (a^{(3)})_1} \\ \frac{\partial C_2}{\partial (a^{(3)})_2} \end{pmatrix} \odot \begin{pmatrix} \sigma'((z^{(3)})_1) \\ \sigma'((z^{(3)})_2) \end{pmatrix} = \begin{pmatrix} (a^{(3)})_1 - (y)_1 \\ (a^{(3)})_2 - (y)_2 \end{pmatrix}.$$

Specify all the dimensions of the matrices/vectors appearing in the backpropagation algorithm applied to this formulation.

Question 7:

You must now modify the code so that it uses a two dimensional output vector. The parts of the code that need to be modified are listed below:

- the size of the matrices/vectors W_3 and b_3 when doing the random initialization
- the backpropagation...if necessary...
- the cost function
- the computation of the accuracy